Intelligent Document Q&A System: A Retrieval-Augmented Generation Pipeline for Document-Grounded Question Answering

Gaurav Bidani | Spring 26
CS 5100 - Fundamentals of Artificial Intelligence

1. Introduction

Organizations accumulate vast quantities of knowledge in the form of internal documents, technical manuals, policy guides, and product documentation. While this information exists in structured formats, retrieving precise answers to natural language questions from these documents remains a fundamentally difficult problem. Traditional keyword-based search systems return lists of potentially relevant documents but do not synthesize an answer, leaving the cognitive burden of reading, interpreting, and combining information entirely on the user. The goal of this project is to build an Intelligent Document Q&A System: a pipeline that accepts user-uploaded documents, indexes their content, and returns accurate, cited natural language answers to user questions.

The core challenge in document question answering is the vocabulary mismatch between user queries and document content. A user asking about feature availability for a specific pricing tier is unlikely to phrase their question using the exact words that appear in the relevant document section, causing traditional keyword-based retrieval to fail or return irrelevant results. This gap between query intent and document vocabulary requires a retrieval approach that operates on semantic meaning rather than surface-level term overlap. Dense neural retrieval addresses this problem by encoding both queries and documents into a shared vector space, where semantically similar content is close regardless of the specific words used.

Beyond retrieval, a second challenge is generation faithfulness. Large language models are capable of producing fluent and confident answers that are not grounded in any retrieved

evidence, a phenomenon commonly referred to as hallucination. For a document question answering system to be reliable and trustworthy in practice, its generated answers must be constrained to the information present in the retrieved context. This requirement motivates the retrieval-augmented generation (RAG) paradigm, in which the language model is provided with retrieved evidence as explicit context and instructed to answer solely from that evidence rather than from its parametric knowledge.

Three peer-reviewed papers define the theoretical foundation of this project. Reimers and Gurevych (2019) introduce Sentence-BERT (SBERT), a modification of the BERT architecture that uses siamese networks with tied weights to produce fixed-size sentence embeddings comparable via cosine similarity. SBERT dramatically reduces the computational cost of large-scale semantic similarity search while maintaining strong performance on semantic textual similarity benchmarks, making it the foundational framework for the embedding approach used in this project.

Karpukhin et al. (2020) propose Dense Passage Retrieval (DPR), which uses two independent BERT-base encoders to embed questions and passages separately, scoring relevance via dot product. Their qualitative analysis reveals that BM25 retains advantages on queries involving rare or highly specific entities, demonstrating that dense and sparse retrieval have complementary strengths -- a finding that directly motivates the hybrid retrieval architecture implemented in this project. Lewis et al. (2020) build on DPR to propose the RAG framework, combining a parametric language model with a non-parametric dense vector index. RAG achieves state-of-the-art results on open-domain question answering benchmarks, with human evaluators judging RAG outputs as more factual than the baseline language model in 42.7% of cases versus only 7.1% for the baseline, validating that grounding generation in retrieved evidence produces more faithful answers.

This project builds on these three papers to construct a practical document Q&A system. It extends the original RAG architecture in several important ways: it adds a cross-encoder re-ranking stage not present in the original RAG system, uses recursive text chunking with overlap rather than fixed disjoint passages, supports user-uploaded documents in multiple formats rather than a static Wikipedia index, and replaces the original DPR encoders and BART generator with modern embedding models and LLM APIs that benefit from substantial advances in model quality since these papers were published.

2. Methods

2.1 System Architecture Overview

The system is implemented as a FastAPI backend with a modular pipeline architecture. It consists of two primary workflows: document ingestion and query answering. During ingestion, uploaded documents are parsed, split into overlapping chunks, embedded into dense vectors, and stored in a persistent vector database. During query answering, the user's question is embedded, candidate chunks are retrieved via hybrid search, optionally re-ranked by a cross-encoder, and passed as context to a language model that generates a cited natural language answer.

The full ingestion pipeline: file upload >> document parsing >> recursive chunking >> embedding >> ChromaDB storage.

The full query pipeline: question embedding >> hybrid retrieval (BM25 + dense, 15 candidates) >> cross-encoder re-ranking (top 5) >> LLM generation with source citation.

2.2 Document Ingestion

Parsing. The system supports PDF, DOCX, and TXT documents using a strategy and factory pattern, where each format has a dedicated parser and a factory function selects the appropriate one based on file extension.

Chunking. Document text is split into overlapping chunks of at most 512 tokens with a 50-token overlap between consecutive chunks. The overlap preserves semantic coherence at chunk boundaries, unlike the fixed disjoint passages used in DPR and the original RAG paper.

Embedding. Each chunk is embedded using BAAI/bge-base-en-v1.5 from the sentence-transformers library, producing a 768-dimensional vector. This model was selected after evaluation showed it outperformed the initially deployed all-MiniLM-L6-v2 model on retrieval recall, as described in the Results section.

Storage. Chunk embeddings, raw text, and source metadata are stored in a ChromaDB persistent vector database using cosine similarity as the distance metric.

2.3 Retrieval

Dense retrieval. User queries are embedded using the same BAAI/bge-base-en-v1.5 model and compared against stored chunk embeddings via cosine similarity in ChromaDB. The top-K results are returned with their similarity scores.

BM25 sparse retrieval. In parallel, BM25 retrieval is performed using the rank-bm25 library. BM25 is a classical term-frequency-based ranking function that scores chunks based on the presence and frequency of query terms. As demonstrated by Karpukhin et al. (2020), sparse retrieval retains advantages over dense retrieval on queries involving exact entity names, numerical values, and rare technical terms that may not be well-represented in the embedding space.

Hybrid score fusion. Dense and sparse retrieval scores are combined using linear score fusion. Both score sets are first normalized to [0, 1] using min-max normalization to make them comparable across different scales. The fused score for each chunk is computed as alpha × dense_score + (1 - alpha) × bm25_score, where alpha = 0.5, giving equal weight to both signals. The top 15 candidates by fused score are retained for optional re-ranking.

Cross-encoder re-ranking. When re-ranking is enabled, the top 15 hybrid candidates are re-scored using the cross-encoder/ms-marco-MiniLM-L-6-v2 model. Unlike the bi-encoder which embeds query and document independently, the cross-encoder processes the query and each candidate chunk jointly, allowing full attention across both. This produces more precise relevance scores at the cost of higher latency, since each (query, chunk) pair requires a separate forward pass. The top 5 chunks by cross-encoder score are passed to the generation stage.

2.4 Generation

Retrieved chunks are formatted into a numbered context block and passed to Google Gemini (gemini-3.1-flash-lite-preview) alongside the user's question. The system prompt instructs the model to answer only from the provided context, cite sources using [Source N] notation, and explicitly state when the context does not contain enough information to answer the question. Temperature is set to 0.2 to encourage factual, deterministic responses. The LLM service supports Google Gemini, OpenAI, and Anthropic APIs via a configuration switch, allowing the underlying model to be changed without modifying the pipeline code.

2.5 Evaluation Methodology

Dataset. The evaluation dataset consists of 20 questions derived from a synthetic internal knowledge base document generated for a fictional company, NovaTech Solutions. The knowledge base was produced using a Claude LLM with targeted prompting to ensure comprehensive coverage of realistic enterprise document content. The question set was similarly generated using iterative prompting to produce questions spanning 16 distinct categories, including single-chunk factual lookup, multi-chunk reasoning, multi-hop arithmetic, negation handling, unanswerable questions, and conversational queries. The dataset was then manually reviewed for accuracy and cross-validated using two additional language models,

ChatGPT and Gemini, to verify that the expected answers were correct and unambiguous. The final dataset includes 7 easy, 9 medium, and 4 hard questions.

Retrieval metric. Retrieval quality is measured using keyword recall. For each question, a set of expected keywords is defined from the ground-truth answer. Recall is the fraction of expected keywords present in the combined text of all retrieved chunks. A recall of 1.0 indicates all expected keywords were retrieved; lower values indicate partial or failed retrieval. Keywords were carefully defined to reflect terms that would appear in source chunks rather than reasoning conclusions, ensuring the metric accurately reflects retrieval coverage.

Generation metrics. Generation quality is measured using an LLM-as-judge approach. For each question, the full RAG pipeline is run to produce a generated answer. This answer is then evaluated by Gemini on two dimensions: faithfulness (1-5), measuring whether the answer is grounded in the retrieved context with no hallucinated claims, and correctness (1-5), measuring how accurately and completely the answer matches the expected answer. This approach follows the methodology established in automated RAG evaluation frameworks and provides interpretable, per-question diagnostic information.

Baselines. Retrieval is evaluated across four configurations: dense only, dense with re-ranking, hybrid without re-ranking, and hybrid with re-ranking. This design isolates the contribution of each system component, allowing a direct comparison of sparse retrieval, dense retrieval, and re-ranking both individually and in combination. Each configuration retrieves an initial candidate pool of 15 chunks before optional re-ranking narrows the set to the top 5 passed to the generation stage. Additionally, all four configurations are evaluated for two embedding models, all-MiniLM-L6-v2 (384 dimensions) and BAAI/bge-base-en-v1.5 (768 dimensions), to quantify the impact of the embedding upgrade independently of the retrieval architecture. This results in a total of eight retrieval configurations evaluated, providing a comprehensive view of how each design decision contributes to overall system performance.

3. Results

3.1 Embedding Model Comparison

The system was initially deployed using the all-MiniLM-L6-v2 embedding model, which produces 384-dimensional vectors and is widely used as a baseline in retrieval systems. During development, the model was upgraded to BAAI/bge-base-en-v1.5, which produces 768-dimensional vectors and is specifically optimized for retrieval tasks. To quantify the impact of this upgrade, both models were evaluated across all four retrieval configurations on the 20-question dataset. The results are presented in the tables below.

Retrieval Recall Comparison

Configuration	MiniLM-L6-v2 Recall	BGE-base Recall	Gain	% Improvement
Dense only	0.8401 (11/20)	0.9010 (13/20)	**0.061**	**+7.3%**
Dense + Rerank	0.9383 (15/20)	0.9383 (15/20)	0	0.0%
Hybrid, no rerank	0.8567 (13/20)	0.9483 (16/20)	**0.092**	**+10.7%**
Hybrid + Rerank	0.9383 (15/20)	0.9383 (15/20)	0	0.0%

Retrieval Latency Comparison

Configuration	MiniLM Time	BGE Time	Change	% Change
Dense only	18.8ms	24.6ms	+5.8ms	**+30.9%**
Dense + Rerank	134.2ms	134.3ms	+0.1ms	+0.1%
Hybrid, no rerank	20.0ms	25.4ms	+5.4ms	**+27%**
Hybrid + Rerank	133.8ms	140.2ms	+6.4ms	+4.80%

The BGE model produces meaningful improvements in configurations that do not use re-ranking. On dense-only retrieval, recall improves by 7.3%, and on hybrid retrieval without re-ranking, recall improves by 10.7%, from 13/20 perfect scores to 16/20. This improvement is attributable to BGE's higher-dimensional representation space and retrieval-specific training objective, which produces embeddings that more accurately capture semantic similarity between queries and passages.

Notably, when re-ranking is enabled, both models converge to identical scores of 0.9383 recall and 15/20 perfect. This indicates that the cross-encoder is strong enough to compensate for weaker initial retrieval, elevating both models to the same level of precision. However, this convergence comes at a significant latency cost of approximately 134ms per query compared to 24-25ms for hybrid retrieval without re-ranking. The BGE model achieves the same re-ranked quality without requiring the re-ranker, making it strictly preferable in latency-sensitive deployments.

On latency, BGE adds approximately 5-6ms overhead on non-reranked configurations due to its larger embedding dimension, representing a 27-31% increase in raw retrieval time. However, in absolute terms this difference is negligible, and the reranked configurations show virtually no difference between the two models since cross-encoder inference dominates total latency. The recall gains from BGE therefore come at a minimal latency cost in practice.

3.2 Retrieval Quality

Using the BGE embedding model, all four retrieval configurations were evaluated. The results are presented below.

Configuration	Avg Recall	Perfect Recall	Avg Retrieval Time
Dense only	0.901	13/20 (65%)	24.6ms
Dense + Rerank	0.9383	15/20 (75%)	134.3ms
Hybrid, no rerank	0.9483	16/20 (80%)	25.4ms
Hybrid + Rerank	0.9383	15/20 (75%)	140.2ms

Hybrid retrieval without re-ranking is the strongest overall configuration, achieving the highest recall (0.9483), the most perfect-recall questions (16/20), and the lowest latency (25.4ms) of all four modes. The hybrid advantage over dense-only retrieval is consistent with the finding of Karpukhin et al. (2020) that BM25 and dense retrieval have complementary strengths: BM25 contributes on exact-match queries involving product names and numerical values, while dense retrieval contributes on semantically paraphrased queries where keyword overlap is low.

Re-ranking improves recall for the dense-only configuration (+0.037) but reduces it for hybrid (-0.010). This is a consequence of the re-ranking candidate pool reduction from 15 to 5: when hybrid search has already assembled a high-quality candidate pool, the cross-encoder occasionally filters out a relevant chunk that ranked highly on one signal but not the other. The tradeoff is that the retained top 5 are more precisely ordered for the generation stage - a benefit that recall alone does not capture.

Four questions did not achieve perfect recall in any configuration. These questions require information that is spread across multiple distant sections of the knowledge base, exceeding the effective retrieval window. Questions Q4 (Standard tier SentinelNet restriction), Q10 (Vertex Gaming event volume comparison), Q15 (GraphQL API production status), and Q17 (multi-hop pricing calculation) all involve facts that are distributed across two or more non-adjacent chunks, making complete retrieval difficult within a fixed top-K budget.

3.3 Generation Quality

Generation quality was evaluated for all 20 questions using the hybrid + rerank configuration with BGE embeddings. All 20 questions were successfully scored.

Metric	Score
Average Faithfulness	5.00 / 5.00
Average Correctness	4.75 / 5.00

Results by difficulty:

Difficulty	Count	Avg Faithfulness	Avg Correctness
Easy	7	5	4.86
Medium	9	5	4.78
Hard	4	5	4.5

Faithfulness scores are perfect across all 20 questions and all difficulty levels. Every claim in every generated answer was directly supported by the retrieved context, with no hallucinated facts observed. This result validates the core design decision of the RAG architecture: by

constraining the language model to answer only from provided context, hallucination is effectively eliminated on this evaluation set.

Correctness scores are strong overall (4.75/5.00) with a gradual degradation as difficulty increases, from 4.86 on easy questions to 4.50 on hard questions. This degradation is consistent with the expectation that harder questions require more complex reasoning across multiple retrieved chunks, which increases the likelihood of incomplete or imprecise answers.

Four questions scored below 5.00 on correctness, each for a distinct reason. Q5 (4/5) was answered correctly but omitted supporting context, a case of correctness without completeness. Q12 (4/5) correctly identified that FedRAMP authorization is not yet complete but introduced ambiguity by reporting conflicting dates found across two retrieved chunks. Q20 (4/5) over-answered by providing the full engineering ladder rather than directly addressing the question. Q17 received the lowest correctness score (3/5) and is discussed in detail below.

The lowest-scoring question is Q17, which asks for a monthly cost estimate for a company with 300 hosts requiring root cause analysis and 30-day data retention. Answering this question correctly requires retrieving pricing from three separate sections, recognizing that 300 hosts exceeds the Professional tier cap of 250 hosts, computing $35 + $12 = $47 per host, and multiplying by 300 to arrive at $14,100 per month. The system correctly retrieved the relevant chunks and correctly reasoned that 300 hosts exceeds the Professional tier limit, but declined to provide a concrete cost estimate, concluding instead that Enterprise pricing is custom. This behavior reflects a known limitation of standard RAG architectures: the system retrieves and presents facts faithfully but does not perform arithmetic reasoning over those facts. The faithfulness score remains 5/5 because the system made no false claims - it accurately reported the constraints it found - but the answer was incomplete relative to what the question required.

The most informative result is the unanswerable question (Q16), which asked about backend programming languages - information not present in the knowledge base. The system correctly

responded that the knowledge base did not contain this information while noting what was available (SDK languages). This demonstrates appropriate uncertainty handling, a property that pure language model systems without retrieval grounding frequently fail to exhibit.

4. Conclusions

4.1 System Strengths

The evaluation results demonstrate that the system achieves its primary design goals. Perfect faithfulness scores across all 20 questions confirm that the RAG architecture reliably prevents hallucination by constraining the language model to retrieved evidence. This is the most important property of a document Q&A system, as users rely on the system's answers to be grounded in the actual content of their documents. The hybrid retrieval architecture consistently outperforms dense-only retrieval, validating the theoretical prediction from Karpukhin et al. (2020) that sparse and dense signals are complementary. The embedding model upgrade from all-MiniLM-L6-v2 to BAAI/bge-base-en-v1.5 produced a measurable 10.7% recall improvement on the most practically relevant configuration, demonstrating that embedding model selection has a significant impact on retrieval quality independent of the retrieval architecture. The system also handles out-of-scope questions appropriately, correctly identifying unanswerable questions and responding with uncertainty rather than a fabricated answer.

4.2 System Weaknesses

The most significant weakness is multi-hop arithmetic reasoning. When a question requires combining facts from multiple chunks and performing a calculation over those facts, the system retrieves the correct evidence but does not complete the reasoning chain. This is a fundamental limitation of the standard RAG architecture, which was designed for retrieval and generation but not for numerical reasoning. The original RAG paper (Lewis et al., 2020) similarly did not address arithmetic tasks, and this limitation has been recognized as a known gap in the RAG literature.

A secondary weakness is the re-ranking recall tradeoff. While the cross-encoder re-ranker improves the precision and ordering of the top results - which benefits generation quality - it reduces raw recall by filtering the candidate pool from 15 to 5. On a small knowledge base such as the one used in this evaluation, the hybrid retriever without re-ranking achieves higher recall at lower latency. In larger knowledge bases where the initial candidate pool contains more noise, the re-ranker's contribution to precision would likely outweigh the recall reduction.

A third limitation is that BM25 index construction occurs on every query, as the index is rebuilt from all stored chunks each time a question is asked. This is not a correctness issue but a performance issue that would become significant at larger document scales. Caching the BM25 index and rebuilding it only when new documents are ingested would eliminate this overhead.

4.3 Comparison to the Original RAG Architecture

This system differs from the original RAG paper (Lewis et al., 2020) in several important ways. The original RAG system operates on a static Wikipedia index, meaning the document collection is fixed and cannot be updated with new content without retraining. This system instead supports arbitrary user-uploaded documents in multiple formats, making it applicable to private and domain-specific knowledge bases. On the generation side, the original RAG system uses a computationally expensive approach that considers multiple retrieved passages simultaneously during answer generation. This system simplifies that process by passing the top retrieved chunks directly as context to the language model, which is more efficient and easier to interpret. Beyond these differences, this system adds two components not present in the original RAG architecture: overlapping text chunking to preserve semantic coherence at chunk boundaries, and a cross-encoder re-ranking stage that improves the precision of the retrieved context before generation.

4.4 Future Work

Several directions would meaningfully improve the system. Tool-augmented generation, in which the language model is given access to a calculator or code execution environment, would address the multi-hop arithmetic weakness by allowing the system to compute rather than retrieve numeric answers. Caching the BM25 index rather than rebuilding it on every query would substantially reduce retrieval latency at document scales beyond the current prototype. Fine-tuning the embedding model on domain-specific question-passage pairs would likely improve retrieval quality beyond what a general-purpose model can achieve. Adding multimodal ingestion support, allowing the system to process images, tables, charts, and scanned documents in addition to plain text, would greatly expand the range of real-world use cases the system can serve, as a significant portion of enterprise knowledge exists in non-textual formats. Finally, a React-based chat interface with inline source citation display would make the system accessible to non-technical users, allowing the pipeline to be evaluated in realistic usage conditions.

Works Cited

Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China. Association for Computational Linguistics.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online. Association for Computational Linguistics.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 9459–9474.

BAAI. 2023. bge-base-en-v1.5: BAAI General Embedding. https://huggingface.co/BAAI/bge-base-en-v1.5

Chroma. 2023. ChromaDB: The AI-native open-source embedding database. https://www.trychroma.com

Hugging Face. 2023. cross-encoder/ms-marco-MiniLM-L-6-v2.
https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2